

Foundations of Programming Languages

Exercises 3

SASyLF & Untyped Lambda-Calculus

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

2026/2027

The exercise numbers in this list refer to the exercise numbers in book *Types and Programming Languages*✶. Benjamin Pierce, The MIT Press, 2002.

Exercises in SASyLF The following exercises are based on file `tapl-nats.slf`, available at the course's web site, and on the proofs in TAPL.

1. Replace all occurrences of **proof** by **unproved** by appropriate proofs, starting with **theorem** `preservation`.
2. Exercises 2 introduced the term `max t1 t2`.
 - (a) Add the new term to the language of terms (remember to declare `max` as a terminal).
 - (b) Add evaluation and typing rules. Remember that we want a left-to-right, call-by-value semantics: we first evaluate `t1` and, only once it is a numeric value `nv1`, do we evaluate `t2`.
 - (c) Extend the proofs of theorems `preservation` and `progress` to account for `max`.

Exercises in TAPL for the untyped lambda-calculus

3. Define the logical functions `or` and `not` (Exercise 5.2.1).
4. Define a term for raising one number to the power of another (Exercise 5.2.4).
5. Adapt the call-by-value rules of Figure 5-3 to describe the other three strategies for evaluation—full beta-reduction, normal-order, and lazy evaluation (Exercise 5.3.6).
6. Show how to formulate the evaluation rules for lambda-terms in the big-step style (Exercise 5.3.8).

Implementation The following exercises can be based on the Haskell source code adapted from TAPL and provided at the course's web site.

7. Write terms for `id`, `true`, `false`, `test`, `and`, `or` and `not`. Note: some of these identifiers are predefined in Haskell's Prelude; the code in `UntypedLambda.hs` hides them with

```
import Prelude hiding (id, and, or, not, fst, snd, succ).
```
8. Write terms for `pair`, `fst`, and `snd`.
9. Write terms for the first few numerals, `c0`, `c1`, ..., as well as functions `succ`, `plus`, `times` and `power`.
10. Complete function `freeVars` in `UntypedLambda.hs`, which computes the set of free variables in a term; function `subs` depends on it. Suggestion: use module `Data.Set`.
11. Write a predicate `isValue` that determines whether a given term is a value.
12. Write a function `step` that performs a single step of call-by-value evaluation. Use function `subs`.
13. Write a function `eval` that repeatedly calls function `step`, until it finds a value.
14. Make sure that the following terms evaluate as in TAPL, Chapter 5, where $v = \lambda x.x\ x$ and $w = \lambda y.y$ (closed values, so that call-by-value evaluation does not get stuck):
 - `id (id (λz.id z))`
 - `test true v w`
 - `and true true`
 - `and true false`
 - `fst (pair v w)`
15. Use `eval` to compute `succ c0`. Is the resulting term equal to `c1`? Discuss.
16. Write a function `bigStep` that implements the big-step semantics of item 6.
17. Compare function `bigStep` against function `eval` by running the examples in item 14 and checking the results.